

cryptography-journal / week06.md

krina808

Update week06.md

abf93dd · 16 minutes ago🕒 History

PreviewCodeBlame

86 lines (57 loc) · 6.95 KB

Raw📄📥📝⋮

Week 06 Journal Activity

Diffie–Hellman Key Exchange Algorithm

The Diffie–Hellman Key Exchange is a cryptographic protocol that allows two parties, who are not mutually knowing one another, to establish a shared secret key over an insecure communication channel. Unlike other encryption algorithms, DHKE is concerned with providing secure means of generating a symmetric key without actually transmitting the key itself. The algorithm is based on the mathematical properties of modular exponentiation and it is considered to be difficult to solve the discrete logarithm problem. DHKE (double headache key exchange) works similarly, both parties choose private key, and produce associated public key with a common prime and a generator. In this protocol both parties exchange the public keys, then compute the shared secret by using its own private key and the received public key. As both calculations give the same result, a symmetric encryption scheme can now be performed using the shared secret for secure communications.

1. Diffie–Hellman in OpenSSL

- Generate DH Parameters: openssl dhparam -out dhparam.pem 2048 This gives me 2048-bit Diffie–Hellman parameters.

2: Generate Private and Public Keys for Party A and Party B

Part A: openssl genpkey -paramfile dhparam.pem -out privateA.pem openssl pkey -in privateA.pem -pubout -out publicA.pem

Part B:

openssl genpkey -paramfile dhparam.pem -out privateB.pem openssl pkey -in privateB.pem -pubout -out publicB.pem

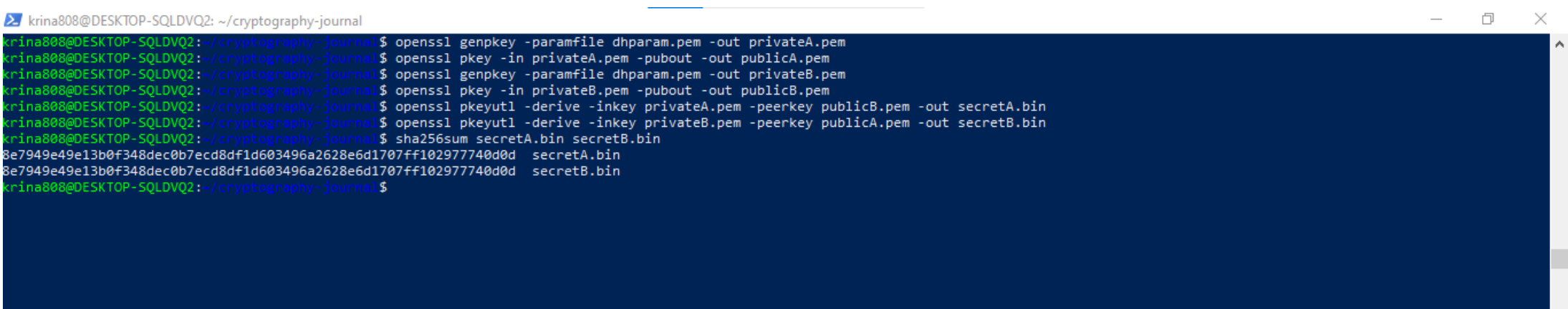
3: Generate Shared Secret

openssl pkeyutl -derive -inkey privateA.pem -peerkey publicB.pem -out secretA.bin

4: Verify the Shared Secrets

sha256sum secretA.bin secretB.bin

Executions:



Diffie–Hellman in Python

Python Code for DHKE

from cryptography.hazmat.primitives.asymmetric import dh from cryptography.hazmat.primitives import serialization

Generate DH parameters

parameters = dh.generate_parameters(generator=2, key_size=2048)

Generate private and public keys for Alice

private_key_a = parameters.generate_private_key() public_key_a = private_key_a.public_key()

Generate private and public keys for Bob

private_key_b = parameters.generate_private_key() public_key_b = private_key_b.public_key()

Exchange public keys and generate shared secret

shared_secret_a = private_key_a.exchange(public_key_b) shared_secret_b = private_key_b.exchange(public_key_a)

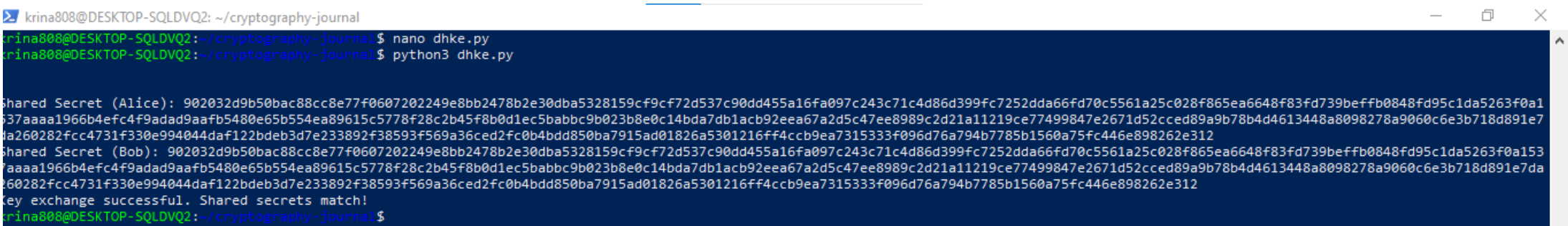
Verify if both shared secrets are equal

print("Shared Secret (Alice):", shared_secret_a.hex()) print("Shared Secret (Bob):", shared_secret_b.hex())

if shared_secret_a == shared_secret_b: print("Key exchange successful. Shared secrets match!") else: print("Key exchange failed. Shared secrets do not match.")

Output:

Shared Secret (Alice):
34ca1283ee179bf1675f31ad14c80ca94bf29c96b2285275efb0f50b25540b42e465e27e830d7627d7b6f3e1c99e81f0b53148f8cd10420ab8623b19
3cc26449576724d3929e963a7393965fb5f2d9f94037b3058bbfb2d8eea5aad1ff67e19cf0e89f392b70af2de5fa183584534ea38aa843cf940c33aa43
e5925d5beae931100aa42d6c3da38e8dcc630bf155a356cada2d0eaa55e016a0e45b1bb2771bfcd7840313958e57c40c54e8cc6c70f7178023101
61d6be3b124a23374373e42377fe31ba453d5b6b3650d8014775ad0d9d1db7cf1f9783f22cf25e1d5bc02dd10d559310a180c4fa4bf5765dff42a3bd
9be381601703ed0570baa89c1b6ac Shared Secret (Bob):
34ca1283ee179bf1675f31ad14c80ca94bf29c96b2285275efb0f50b25540b42e465e27e830d7627d7b6f3e1c99e81f0b53148f8cd10420ab8623b19
3cc26449576724d3929e963a7393965fb5f2d9f94037b3058bbfb2d8eea5aad1ff67e19cf0e89f392b70af2de5fa183584534ea38aa843cf940c33aa43
e5925d5beae931100aa42d6c3da38e8dcc630bf155a356cada2d0eaa55e016a0e45b1bb2771bfcd7840313958e57c40c54e8cc6c70f7178023101
61d6be3b124a23374373e42377fe31ba453d5b6b3650d8014775ad0d9d1db7cf1f9783f22cf25e1d5bc02dd10d559310a180c4fa4bf5765dff42a3bd
9be381601703ed0570baa89c1b6ac Key exchange successful. Shared secrets match!



Systems and Explanation of Success/Failure: Illustrating Attacks

The man in the middle (MITM) attack is a common way to attack systems where an attacker intercepts communication between two parties and may change or listen to their messages. In the case of Diffie–Hellman key exchange the attacker relies on an MITM attack, which is successful if the attacker can intercept public keys exchanged between the parties and substituting those with its own public keys. In this case, the attacker would be able to derive two separate shared secrets with each party. Nevertheless, this attack will fail if the parties involved both authentic communication parties (e.g., with digital certificates or out of band proof) before initiating the key exchange. Authentication of MITM attack eliminates compromised security, while without authentication it can be fatal.

Difficulties in Understanding Security Systems and Their Relation

The one difficulty I had with understanding security systems in the case of Diffie–Hellman key exchange was indeed hard to grasp mathematically, how the algorithm worked: the mod exponentiation, and the discrete logarithms problem. Another thing that was challenging to understand was the interrelationship of security protocols together (e.g. How DHKE supports having encryption systems like AES active in secure communication), being dependent of underlying mathematical knowledge combined of thorough but complex practice.

Links to and Short Summaries of Websites/Papers/Software on Security Systems and Their Attacks

OWASP (Open Web Application Security Project) – OWASP Top Ten link: <https://owasp.org/www-project-top-ten/>

This is a complete list of the 10 most critical web application security risks. It specifies vulnerabilities, including SQL injection, cross site scripting (XSS) as well as broken authentication. Therefore it also describes how these vulnerabilities are exploited in web applications and how to mitigate them.

Insights & Reflections

The Diffie–Hellman Key Exchange (DHKE) algorithm is a secure affair for two parties to establish a shared secret over an insecure channel, the basis for cryptography. DHKE prevents an attacker from obtaining the shared secret even if intercepted values are used by creating a shared secret that is dependent on large prime numbers and using the mathematical properties of modular arithmetic such that even after intercepting the exchanged values there is no easy way to derive the shared secret while solving the discrete logarithm problem. When it came to implementing the algorithm on Python, I realized how both parties create their public keys and what a common secret can be for both using each other's public key. It was evident that readers must pick prime numbers and private keys safely, poor choices could compromise the system to an attack. Moreover, it illustrated the possibility of a man-in-the-middle attack and also the necessity of authentication mechanisms to avoid such a risk. The whole procedure of implementing DHKE helped me understand secure key exchange protocols better and what they are for in the world of modern cryptography.

